Reviewer Pack — Minimal Rank of Noncommutative Crossed Products vs BP_ReadTw…

Entry cc84f07e5cd1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 21:36:43 UTC

Minimal Rank of Noncommutative Crossed Products vs BP_ReadTwice Circuit Threshold for k-CNF

Entry ID: `cc84f07e5cd1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 21:36:43 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry and Algebra (Noncommutative Crossed Products) **Field B** (complexity object): Complexity Theory (BP_ReadTwice Circuit Complexity)

Statement:

> {'text': 'For every instance of k-CNF with n ≤ 40 variables, the minimal rank of the noncommutative crossed product associated with the clause indicator polynomial is Θ(log(n)) lower than the BP_readtwice circuit threshold.', 'mathematical': 'Let I be an instance of k-CNF with n variables. Then, ρ_{nc}(I) ≤ β(log(n)), where ρ_{nc}(I) is the minimal rank of the noncommutative crossed product associated with the clause indicator polynomial of I, and β is a constant.', 'counterexample': 'For an instance of k-CNF with n variables, if the associated noncommutative crossed product has a rank lower than Θ(log(n)), then the BP_readtwice circuit threshold for that instance is not greater than ρ_{nc}(I) + β(log(n)).'}

Rationale (proposer's reasoning):

> {'text': 'Noncommutative crossed products have been used to model certain algebraic structures in physics and geometry. Their ranks might capture inherent complexities in the clause indicator polynomials of k-CNF instances, potentially explaining gaps between BP_readtwice circuit thresholds.', 'mathematical': 'The noncommutative crossed product formalism could provide a new tool for understanding the complexity of Boolean functions and their polynomial representations.'}

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e559f4723b5266c1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≤ 40, the average difference between the minimal rank of noncommutative crossed products and BP_readtwice circuit thresholds is less than

or equal to β * log(n) with a significance level of $p < 0.05$ based on at least 30 independent trials.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'noncommutative geometry' AND 'crossed product' AND 'BP_readtwice complexity'` - `'minimal rank' AND 'noncommutative crossed products' AND 'k-CNF circuit threshold'` - `'clause indicator polynomial' AND 'noncommutative algebra' AND 'BP_readTwice circuit'`

Top relevant hits considered: - [http://arxiv.org/abs/math/0610043v5] Lecture Notes on Noncommutative Algebraic Geometry and Noncommutative Tori - [http://arxiv.org/abs/0705.1265v2] A noncommutative Bohnenblust-Spitzer identity for Rota-Baxter algebras solves Bogoliubov's recursion - [http://arxiv.org/abs/1310.5673v2] The Bell states in noncommutative algebraic geometry - [http://arxiv.org/abs/2002.05617v3] Waring rank of binary forms, harmonic cross-ratio and golden ratio - [http://arxiv.org/abs/2005.00148v4] The Stable Rank of Diagonal ASH Algebras and Crossed Products by Minimal Homeomorphisms - [http://arxiv.org/abs/2107.00223v2] Exponential Lower Bounds for Threshold Circuits of Sub-Linear Depth and Energy - [http://arxiv.org/abs/1606.00596v3] Randomized Polynomial Time Identity Testing for Noncommutative Circuits - [http://arxiv.org/abs/1905.00129v1 Knuth-Bendix Completion Algorithm and Shuffle Algebras For Compiling NISQ Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(cols):
            if j != i:
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix
```

```python
def rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rref_matrix = gaussian_elimination(matrix)
    rank = 0
    for row in rref_matrix:
        if any(row):
            rank += 1
    return rank

def clause_indicator_polynomial(cnf_instance, n):
    m = len(cnf_instance)
    polynomial = [[0] * (2**n) for _ in range(m)]
    for i, clause in enumerate(cnf_instance):
        for literal in clause:
            index = sum(1 << j if x == -j-1 else 0 for j, x in enumerate(literal))
            polynomial[i][index] += 1
    return polynomial

def bp_readtwice_circuit_threshold(n):
    # Placeholder function to compute the BP_readtwice circuit threshold
    # This is a dummy implementation and should be replaced with actual logic
    return n * math.log2(n)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    cnf_instance = [[random.choice([-i-1, i]) for _ in range(random.randint(1, 3))] for _ in range

    polynomial = clause_indicator_polynomial(cnf_instance, n)
    nc_rank = rank(polynomial)
    bp_threshold = bp_readtwice_circuit_threshold(n)

    difference = bp_threshold - nc_rank

    return {
        "metric_name": "Rank vs BP_ReadTwice",
        "metric_value": difference,
        "instances_tested": 1,
        "conjecture_holds": difference >= math.log2(n),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30))  # Default to first 30 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_faili
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_904dfdd2.py", line 78, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_904dfdd2.py", line 56, in run_trial
    cnf_instance = [[random.choice([-i-1, i]) for _ in range(random.randint(1, 3))] for _ in range(n)]
                                    ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11783
2	propose	ollama_remote	glm4:latest	0	0	10677
3	propose	ollama_remote	glm4:latest	0	0	12305
4	preregistration	ollama_remote	glm4:latest	0	0	5692
5	novelty	ollama_remote	glm4:latest	0	0	4900
6	novelty	ollama_remote	glm4:latest	0	0	6516
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18222
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9807

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13685
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9323
11	judge	ollama_remote	glm4:latest	0	0	10656

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 113564 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/cc84f07e5cd1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cc84f07e5cd1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cc84f07e5cd1.tar.gz` (if generated)